

Phase -1.14 — Final Architecture Review & Implementation Roadmap

Objective

This marks the completion of the planning phase.

Before writing a single line of production code, the engineering team should verify that the architecture, requirements, workflows, and engineering standards are complete.

The objective of this phase is to answer one question:

Are we confident enough in the design that implementation can begin without major architectural changes?

A mature engineering team spends significant time reviewing architecture before implementation because changes become exponentially more expensive as development progresses.

Planning Phase Recap

At this point, the following artifacts have been completed.

Phase	Deliverable
Phase -1.1	Product Vision
Phase -1.2	Functional Requirements
Phase -1.3	Non-Functional Requirements
Phase -1.4	Actors, Roles & User Stories
Phase -1.5	High-Level Design
Phase -1.6	Low-Level Design
Phase -1.7	Architecture Decision Records
Phase -1.8	API Design & REST Contract
Phase -1.9	Database Design & ER Diagram
Phase -1.10	Sequence Diagrams
Phase -1.11	Capacity Planning

Phase	Deliverable
Phase -1.12	Git Strategy & Development Workflow
Phase -1.13	Coding Standards

These documents together form the system blueprint.

Architecture Review Checklist

Before implementation begins, each item below should be reviewed.

Product

- Are the business goals clearly defined?
 - Is the project scope realistic?
 - Are future extensions identified?
-

Functional Requirements

- Are all required features documented?
 - Are acceptance criteria defined?
 - Are business rules unambiguous?
-

Non-Functional Requirements

- Performance
- Security
- Reliability
- Maintainability
- Scalability
- Observability
- Deployability

Every quality attribute should have measurable expectations.

API Design

Review the following.

- Endpoint naming

- HTTP methods
- Status codes
- Authentication
- Authorization
- Validation
- Response format
- Error format
- Versioning

The API contract should be considered stable before implementation.

Database Review

Verify:

- Entities
- Relationships
- Primary keys
- Foreign keys
- Indexes
- Constraints
- Audit fields
- Soft delete strategy

Database redesigns become increasingly expensive once development begins.

Security Review

Confirm that the architecture includes:

- Password hashing
- JWT authentication
- Refresh token rotation
- RBAC
- Environment variable management
- Secret isolation
- Global error handling
- Input validation

Security should be built into the architecture rather than added later.

Infrastructure Review

Current infrastructure stack.

GitHub

↓

GitHub Actions

↓

Docker Compose

↓

Node.js

↓

Express

↓

Prisma

↓

Supabase PostgreSQL

↓

Redis

↓

BullMQ

↓

Pino

↓

Prometheus

↓

Grafana

Every component should have a clearly defined responsibility.

Development Environment

The implementation environment should now be fully defined.

Local Development

- Ubuntu
- VS Code
- Docker Desktop / Docker Engine
- Docker Compose
- Node.js
- Git
- Postman
- PostgreSQL (Supabase)
- Local Redis
- Local Prometheus
- Local Grafana

Production Simulation

Although the project uses free infrastructure, the architecture should closely resemble production.

Examples:

- Docker containers instead of local processes.
- Reverse proxy instead of direct application exposure.
- Local Redis instead of managed Redis.
- Local monitoring stack.
- GitHub Actions instead of manual builds.

The goal is production parity, not production cost.

Risk Assessment

Every project has technical risks.

High Risk

- Seat booking concurrency.
 - Database transactions.
 - Redis locking.
 - Background job reliability.
-

Medium Risk

- Authentication.
 - Refresh token rotation.
 - Cache invalidation.
 - Docker networking.
-

Low Risk

- CRUD APIs.
- Event browsing.
- Documentation.
- Logging.

The highest-risk components should be implemented and tested early.

Implementation Strategy

The project should be developed incrementally.

Each phase must leave the repository in a deployable state.

Recommended workflow:

Plan

↓

Implement

↓

Test



Skipping any step creates technical debt.

Milestone Roadmap

Milestone 1

Foundation

- Repository
- Docker
- TypeScript
- Express
- Configuration
- Logging
- GitHub Actions

Milestone 2

Authentication

- JWT
- Refresh Tokens
- RBAC

- User Module
-

Milestone 3

Events

- Event Management
 - Seat Management
 - Redis Cache
-

Milestone 4

Bookings

- Transactions
 - Concurrency Control
 - BullMQ
 - Seat Reservation
-

Milestone 5

Payments

- Mock Payment Provider
 - Booking Confirmation
 - Background Jobs
-

Milestone 6

Production Readiness

- Monitoring
 - Metrics
 - Load Testing
 - Documentation
 - CI/CD
 - Security Hardening
-

Engineering Rules

From this point onward, every implementation must satisfy the following requirements.

No shortcut should violate:

- Architecture
- Security
- Code Quality
- Testing
- Documentation
- Git Workflow
- Coding Standards

If an implementation conflicts with the architecture, the architecture should be reviewed rather than bypassed.

Phase 0 Preview

The next phase transitions from planning to implementation.

Phase 0 is not simply "Project Setup."

It establishes the engineering foundation upon which every future feature depends.

Topics include:

- Installing required software.
- Repository creation.
- GitHub repository configuration.
- Branch protection.
- GitHub Project (Kanban).
- Issue templates.
- Pull Request templates.
- Repository labels.
- Semantic versioning.
- Release strategy.
- Docker installation.
- Node.js installation.
- VS Code configuration.
- Development tooling.
- Project initialization.
- TypeScript configuration.
- ESLint.

- Prettier.
- Husky.
- lint-staged.
- Environment configuration.
- Docker Compose.
- Initial GitHub Actions workflow.

No business features will be implemented during Phase 0.

The goal is to create an engineering environment that mirrors how a production backend project begins.

Deliverables

At the completion of Phase -1, the project has:

- A complete product vision.
- Functional and non-functional specifications.
- High-Level and Low-Level architecture.
- Architecture Decision Records.
- API contract.
- Database design.
- Sequence diagrams.
- Capacity planning.
- Git workflow.
- Coding standards.
- Production engineering guidelines.
- Implementation roadmap.

The planning phase is now complete.

Phase 0 — Engineering Foundation & Project Setup begins the implementation journey, where every architectural decision documented here will be translated into production-quality code and infrastructure.